

گزارش جامع طراحی، معماری و پیاده‌سازی سامانه

CineTrack

مدل ساده و مدل پیشرفته

معماری کلاینت و سرور، طراحی پایگاه داده و مستندسازی رابط‌های سامانه

طراحی و پیاده‌سازی سامانه CineTrack	عنوان پروژه
Flutter / Dart -- FastAPI / Python -- PostgreSQL -- Redis -- TMDB	فناوری‌های اصلی
مدل ساده محلی و مدل پیشرفته کلاینت و سرور	معماری
Flutter + TMDB + SQLite	نسخه ساده
Flutter + Nginx + FastAPI + PostgreSQL + Redis + TMDB	نسخه پیشرفته
برنامه‌سازی موبایل	نام درس
آقای نجیمی	نام استاد
امیرهمایون شریفی‌زاده، ارشیا ایزدیاری، فرزاد کوهی رونقی	اعضای تیم
401106403, 401105656, 401106114	شماره دانشجویی

چکیده

سامانه CineTrack یک برنامه موبایل برای جست‌وجو و مشاهده اطلاعات فیلم و سریال، مدیریت وضعیت تماشا، ثبت قسمت‌های دیده‌شده، امتیازدهی، نظر، علاقه‌مندی‌ها، فهرست‌های شخصی و آمار فعالیت کاربر است. پروژه در دو معماری مستقل پیاده‌سازی شده است: **مدل ساده** که در آن اپلیکیشن Flutter مستقیماً با سرویس اطلاعات فیلم و سریال ارتباط برقرار می‌کند و داده‌های شخصی را در SQLite محلی نگه می‌دارد؛ و **مدل پیشرفته** که یک معماری کامل کلاینت/سرور شامل Flutter، درگاه امن Nginx، بک‌اند FastAPI، پایگاه داده PostgreSQL، کش Redis و آداپتور سرویس TMDB دارد.

فهرست مطالب

i	چکیده
1	1 منابع مورد بررسی
1	1.1 منابع طراحی داخل مخزن
2	2 شمای کلی مسئله و قابلیت‌های سامانه
2	2.1 بازیگران اصلی
3	3 مدل ساده: طراحی، معماری و نحوه کار
3	3.1 فلسفه مدل ساده
4	3.2 نقطه شروع برنامه
4	3.3 لایه‌های کد در مدل ساده
5	3.3.1 Repository به‌عنوان مرکز هماهنگی
5	3.4 ارتباط مستقیم با TMDB
5	3.4.1 مسیرهای اصلی TMDB در مدل ساده
5	3.5 پایگاه داده محلی مدل ساده
6	3.6 احراز هویت و نشست در مدل ساده
7	3.7 بازیابی رمز در مدل ساده
7	3.8 فلو‌های مهم در نرم افزار
7	3.8.1 صفحه اصلی
7	3.8.2 جست‌وجو
7	3.8.3 جزئیات فیلم/سریال
7	3.8.4 امتیاز، نظر، کتابخانه و فهرست‌ها
7	3.9 کش تصویر و پاک‌سازی امن
8	4 مدل پیشرفته و معماری آن

8	4.1 معماری کلان
9	4.2 اصل مالکیت داده
9	4.3 لایه‌های سمت کلاینت پیشرفته
10	4.4 لایه‌های سمت سرور
11	4.5 چرخه یک درخواست در بک‌اند
11	4.6 انتزاع ارائه‌دهنده اطلاعات رسانه‌ای
12	4.7 نحوه استفاده از TMDb در سرور
12	4.8 کش چندلایه در مدل پیشرفته
12	4.9 پنل مدیریت

5 مستندات API مدل پیشرفته 13

13	5.1 اصول قرارداد API
13	5.2 آدرس پایه و پیکربندی کلاینت
13	5.3 احراز هویت API
14	5.4 صفحه‌بندی و فیلتر
14	5.5 قالب خطای استاندارد
14	5.6 Idempotency-Key
15	5.7 تصاویر
15	5.8 فهرست کامل endpointها
18	5.9 نمونه‌های درخواست و پاسخ
18	5.9.1 ورود
18	5.9.2 جست‌وجو
19	5.9.3 ثبت امتیاز
19	5.9.4 ثبت نظر اسپویلر

6 اتصال اپلیکیشن Flutter به سرور در مدل پیشرفته 20

20	6.1 اصل ارتباط
20	6.2 پیکربندی زمان ساخت
20	6.3 پین‌کردن گواهی TLS
20	6.4 ذخیره امن توکن
21	6.5 افزودن خودکار Authorization
21	6.6 تمدید خودکار نشست
22	6.7 Repository در کلاینت پیشرفته

22	6.8 دریافت امن تصویر
23	7 طراحی پایگاه داده و مالکیت اطلاعات
25	8 امنیت
25	8.1 مدل ساده
25	8.2 مدل پیشرفته
25	8.2.1 رمز و نشست
25	8.2.2 مجوز
25	8.2.3 انتقال امن
26	8.2.4 کلید سرویس بیرونی
26	8.2.5 اعتبارسنجی و ورودی مخرب
26	8.2.6 محافظت SSRF در پوستر
26	8.2.7 limiting Rate و هدرهای امنیتی
27	9 قابلیت اطمینان، کارایی، مقیاس پذیری و مصرف منابع
27	9.1 قابلیت اطمینان
27	9.2 مقیاس پذیری
27	9.3 مصرف باتری و اینترنت
28	10 مقایسه مدل ساده و پیشرفته
29	11 محدودیت ها و چالش ها
29	11.1 TMDB به جای IMDb
29	11.2 کش جست و جو
30	12 جمع بندی

فهرست جداول

2	2.1 گروه‌های اصلی قابلیت‌های CineTrack
4	3.1 لایه‌های مدل ساده
6	3.2 جداول SQLite در مدل ساده
9	4.1 ساختار کلاینت در مدل پیشرفته
10	4.2 ساختار بک‌اند در مدل پیشرفته
15	5.1 فهرست endpoint‌های مدل پیشرفته

فهرست تصاویر

3	3.1 معماری کلان مدل ساده
8	4.1 معماری کلان مدل پیشرفته
11	4.2 چرخه پردازش درخواست در مدل پیشرفته
21	6.1 جریان نوسازی خودکار نشست در کلاینت

فصل 1

منابع مورد بررسی

1.1 منابع طراحی داخل مخزن

مهم‌ترین مستندات طراحی موجود در مدل ساده عبارت‌اند از:

- docs/ARCHITECTURE.md •

- docs/SIMPLE_MODEL_AUDIT.md •

- docs/LOCAL_DATABASE.md •

- docs/TMDB_SETUP.md •

و در مدل پیشرفته، علاوه بر موارد فوق، مستندات تخصصی زیر وجود دارد:

- docs/API_CONNECTION.md •

- docs/API_EXAMPLES.md •

- docs/DATABASE.md •

- docs/SECURITY.md •

- docs/TESTING.md •

- docs/DEPLOYMENT_UBUNTU.md •

- docs/OPERATIONS.md •

- docs/openapi.json •

فصل 2

شمای کلی مسئله و قابلیت‌های سامانه

2.1 بازیگران اصلی

سامانه از دید مفهومی سه نوع بازیگر دارد:

مهمان: جست‌وجو، مشاهده صفحه اصلی، اطلاعات فیلم/سریال، فصل‌ها، قسمت‌ها و نظرات عمومی.

کاربر ثبت‌نام‌شده: همه قابلیت‌های مهمان به‌علاوه کتابخانه تماشا، قسمت‌های دیده‌شده، امتیاز، نظر، علاقه‌مندی، فهرست شخصی، پروفایل، آمار و تاریخچه فعالیت.

مدیر: در مدل پیشرفته، علاوه بر قابلیت‌های کاربر، مدیریت کاربران، گزارش‌ها، نظرات نامناسب، داده‌های کش‌شده و آمار کلی سیستم.

جدول 2.1: گروه‌های اصلی قابلیت‌های CineTrack

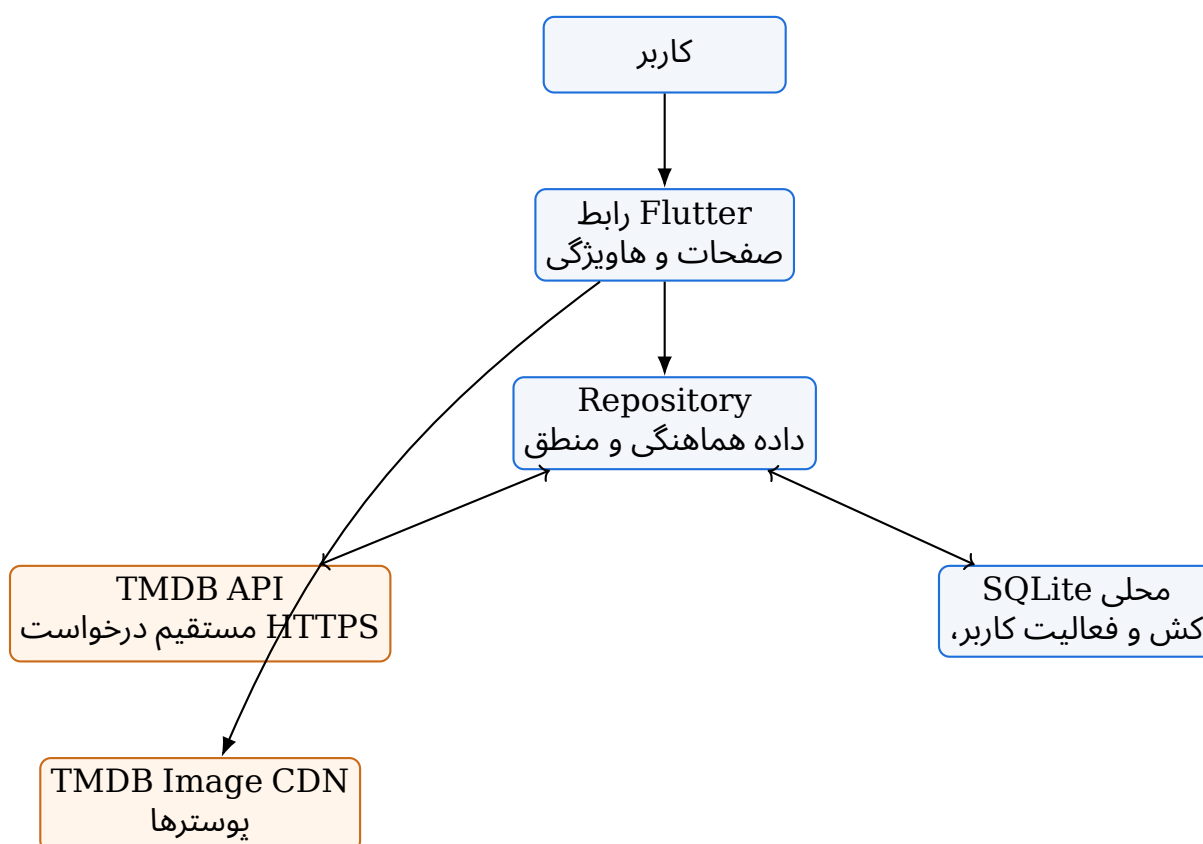
گروه	قابلیت‌ها
حساب کاربری	ثبت‌نام، ورود، خروج، بازیابی رمز، پروفایل، تصویر کاربر، نشست و بیومتریک
کاتالوگ	صفحه اصلی، جست‌وجو، فیلتر بازیگر/کارگردان/ژانر/سال، جزئیات فیلم و سریال
سریال	فصل‌ها، قسمت‌ها، علامت دیده‌شدن، تعداد دیده‌شده/باقی‌مانده و درصد پیشرفت
تعامل کاربر	وضعیت تماشا، علاقه‌مندی، امتیاز ۱ تا ۵، نظر و اسپویلر
سازمان‌دهی	فهرست‌های شخصی و اضافه/حذف آثار
گزارش‌دهی	آمار کاربر، تاریخچه فعالیت و در مدل پیشرفته گزارش تخلف و مدیریت

فصل 3

مدل ساده: طراحی، معماری و نحوه کار

3.1 فلسفه مدل ساده

هدف مدل ساده حداقل کردن زیرساخت است. هیچ سرور اختصاصی، پایگاه داده مرکزی، JWT، Redis یا Nginx وجود ندارد. اپلیکیشن موبایل هم نقش رابط کاربری و هم نقش منطق برنامه و نگهداری داده شخصی را بر عهده دارد. اطلاعات فیلم و سریال مستقیماً از TMDB دریافت می‌شود و داده کاربر روی همان دستگاه ذخیره می‌شود.



شکل 3.1: معماری کلان مدل ساده

3.2 نقطه شروع برنامه

ورودی برنامه در `mobile/lib/main.dart` قرار دارد. روند شروع به ترتیب چنین است:

1. Flutter binding مقداردهی می‌شود.
2. `AppConfig.validateForStartup()` وجود توکن خواندن TMDB را کنترل می‌کند.
3. پایگاه داده محلی با `LocalDatabase.instance.initialize()` باز می‌شود.
4. برنامه داخل `ProviderScope` اجرا می‌شود تا Riverpod وابستگی‌ها و وضعیت را مدیریت کند.

در `mobile/lib/app.dart`، برنامه به زبان فارسی و راست‌به‌چپ پی‌کربندی شده و `AuthGate` بر اساس وضعیت نشست تصمیم می‌گیرد صفحه ورود، صفحه قفل بیومتریک، اسپلش یا پوسته اصلی نشان داده شود.

3.3 لایه‌های کد در مدل ساده

جدول 3.1: لایه‌های مدل ساده

مسئولیت	مسیر
صفحه‌ها، فرم‌ها و جریان‌های تعامل کاربر	<code>mobile/lib/features/*</code>
هماهنگ‌سازی منطق کاربردی، SQLite و TMDB	<code>features/shared/repository.dart</code>
ارسال مستقیم درخواست به TMDB، کش پاسخ و حذف درخواست هم‌زمان تکراری	<code>core/api/tmdb_client.dart</code>
ساخت و نگهداری شمای SQLite	<code>core/storage/local_database.dart</code>
نگهداری نشست محلی و تاریخ انقضا	<code>core/storage/session_store.dart</code>
هش رمز با PBKDF2-HMAC-SHA256 و نمک تصادفی	<code>core/security/password_hasher.dart</code>
بازکردن نشست معتبر با اثرانگشت/چهره	<code>core/auth/biometric_auth.dart</code>
پاک‌سازی امن کش بدون حذف داده شخصی	<code>core/storage/cache_maintenance.dart</code>

3.3.1 Repository به عنوان مرکز هماهنگی

بزرگ‌ترین بخش مدل ساده `mobile/lib/features/shared/repository.dart` است. این کلاس بین سه جهان ارتباط برقرار می‌کند: مدل‌های نمایش، داده‌های شبکه و داده‌های محلی. برای نمونه، جست‌وجوی عنوان از TMDb انجام می‌شود، نتایج به `MediaSummary` تبدیل می‌شوند و خلاصه رسانه برای استفاده‌های بعدی در `media_cache` ذخیره می‌شود. در مقابل، عملیات کاربر مانند امتیاز یا نظر مستقیماً روی جداول محلی انجام می‌شود.

3.4 ارتباط مستقیم با TMDb

کلاینت شبکه در `mobile/lib/core/api/tmdb_client.dart` با `Dio` ساخته شده است. توکن TMDb از `dart-define` دریافت و در هدر `Authorization: Bearer ...` قرار می‌گیرد. زمان اتصال/دریافت/ارسال نیز محدود شده است.

3.4.1 مسیرهای اصلی TMDb در مدل ساده

از روی `Repository`، مسیرهای مهم عبارت‌اند از:

- `/movie/popular` و `/tv/popular` برای آثار محبوب؛
- `/movie/now_playing` و `/tv/on_the_air` برای انتشارهای جدید؛
- `/movie/topRated` و `/tv/topRated` برای آثار برتر؛
- `/trending/all/week` برای پیشنهادهای ترند؛
- `/search/movie`، `/search/tv` و `/discover/*` برای جست‌وجو و فیلتر؛
- `/search/person` و `/person/{id}/combined_credits` برای بازیگر/کارگردان؛
- `/movie/{id}` و `/tv/{id}` برای جزئیات؛
- `/tv/{id}/season/{n}` برای قسمت‌های فصل.

3.5 پایگاه داده محلی مدل ساده

شما در `mobile/lib/core/storage/local_database.dart` ساختار می‌شود. `foreign_keys` فعال است و حالت ژورنال WAL برای بهبود رفتار هم‌زمانی استفاده می‌شود.

جدول 3.2: جداول SQLite در مدل ساده

جدول	کاربرد
users	حساب محلی، مشخصات، هش و نمک رمز، مسیر آواتار، تاریخها
password_reset_tokens	توکن بازیابی محلی با زمان انقضا
media_cache	خلاصه و جزئیات کش شده رسانه با شناسه TMDB
api_cache	کش عمومی پاسخ های TMDB بر اساس کلید درخواست
user_media	وضعیت تماشا و پرچم علاقه مندی برای هر کاربر/رسانه
ratings	امتیاز ۱ تا ۵؛ کلید اصلی مرکب کاربر/رسانه مانع امتیاز تکراری می شود
comments	نظر، اسپویلر، زمان ایجاد/ویرایش
custom_lists	فهرست های شخصی
custom_list_items	عضویت رسانه در فهرست
user_episodes	قسمت های دیده شده و زمان تقریبی
activity	تاریخچه فعالیت کاربر
reports	گزارش نظر در مدل محلی

3.6 احراز هویت و نشست در مدل ساده

در این معماری «احراز هویت سروری» وجود ندارد. هنگام ثبت نام، رمز با نمک تصادفی ۱۶ بایتی و PBKDF2-HMAC-SHA256 با ۴۰ هزار تکرار هش می شود و فقط هش و نمک ذخیره می شوند. هنگام ورود، رکورد ایمیل از SQLite خوانده و هش به صورت مقایسه ثابت زمان تقریبی بررسی می شود.

نشست در SessionStore با SharedPreferences شامل شناسه کاربر و تاریخ انقضا ذخیره می شود. نسخه فعلی بعد از ورود موفق، نشست ۳۰ روزه ایجاد می کند. بیومتریک فقط قفل محلی روی همین نشست معتبر است و هویت شبکه ای جدید ایجاد نمی کند. چون تمام حساب ها روی یک دستگاه هستند، حساب کاربری مدل ساده «حساب سراسری سرویس» نیست و بین چند دستگاه همگام نمی شود.

3.7 بازیابی رمز در مدل ساده

درخواست فراموشی رمز ابتدا ایمیل را در جدول users پیدا می‌کند، سپس یک توکن محلی ۱۵ دقیقه‌ای تولید و در password_reset_tokens ذخیره می‌کند. برنامه همان دستگاه می‌تواند به صفحه بازنشانی هدایت شود. این روش از نظر جریان نرم‌افزاری کامل است، اما ایمیل واقعی ارسال نمی‌شود زیرا مدل ساده عمداً سرور/سرویس ایمیل ندارد.

3.8 فلوهای مهم در نرم افزار

3.8.1 صفحه اصلی

هفت درخواست مستقل به‌طور هم‌زمان با Future.wait ارسال می‌شوند. نتایج فیلم و سریال برای بخش‌هایی مانند «جدیدها» و «برترین‌ها» ادغام و مرتب می‌شوند. هر درخواست ۳۰ دقیقه کش دارد. این طراحی زمان انتظار را نسبت به ارسال سریالی درخواست‌ها کاهش می‌دهد.

3.8.2 جست‌وجو

اگر نام بازیگر یا کارگردان داده شود، ابتدا شخص در TMDb پیدا می‌شود و سپس از combined_credits آثار او استخراج می‌گردد. در غیر این صورت از search یا discover استفاده می‌شود. فیلتر نوع، سال و ژانر در سطح درخواست یا پس‌پردازش اعمال می‌شود.

3.8.3 جزئیات فیلم/سریال

شناسه داخلی مدل ساده به نوع رسانه و شناسه TMDb تجزیه می‌شود. جزئیات کامل با credits/external_ids دریافت و تا ۲۴ ساعت کش می‌شود. برای سریال، فصل‌ها جداگانه دریافت می‌شوند تا قسمت‌ها نیز قابل علامت‌گذاری باشند.

3.8.4 امتیاز، نظر، کتابخانه و فهرست‌ها

این عملیات شبکه‌ای نیستند. Repository با دستورات پارامتری‌شده SQLite رکوردها را درج، جایگزین یا حذف می‌کند. کلیدهای مرکب و محدودیت CHECK از بسیاری از تکرارها و امتیاز خارج از محدوده جلوگیری می‌کنند.

3.9 کش تصویر و پاک‌سازی امن

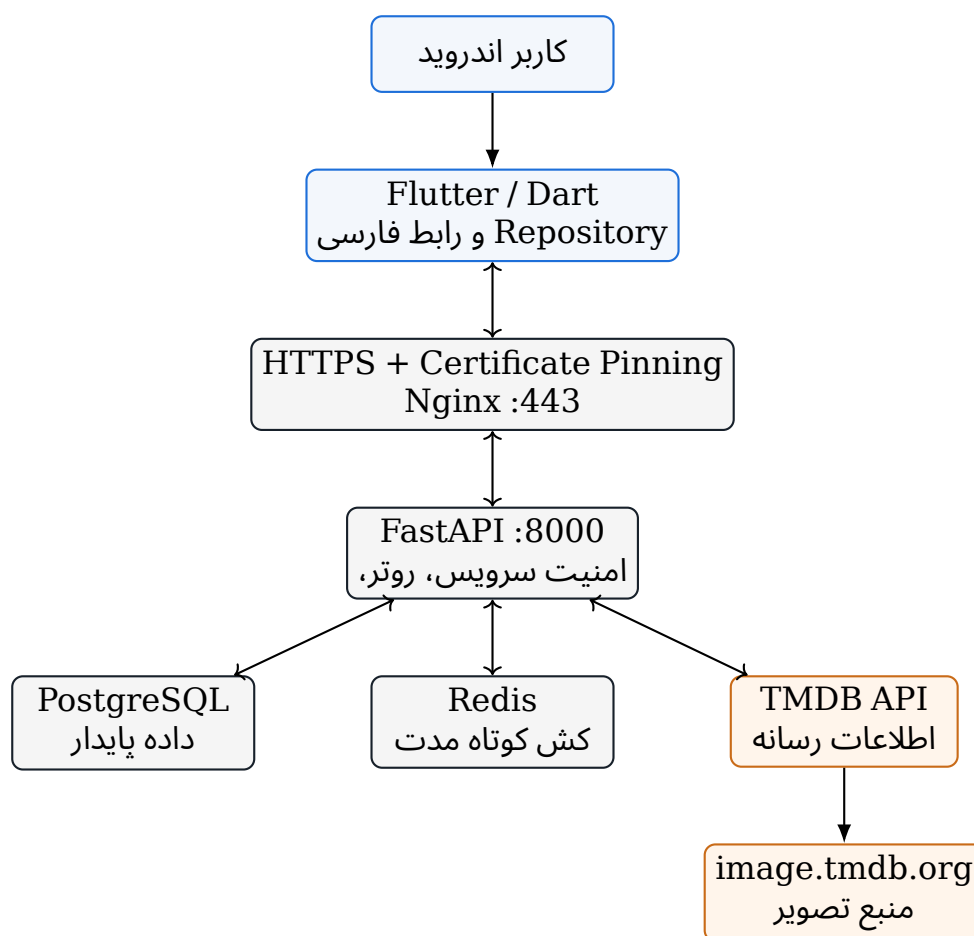
پوسترهای شبکه با cached_network_image و flutter_cache_manager روی دستگاه کش می‌شوند.

فصل 4

مدل پیشرفته و معماری آن

4.1 معماری کلان

مدل پیشرفته مسئولیت‌ها را از دستگاه جدا می‌کند. کلاینت دیگر مستقیماً به TMDB وصل نمی‌شود. همه درخواست‌ها به CineTrack API می‌روند و بک‌اند مسئول احراز هویت، مجوز، پایگاه داده، ارتباط با سرویس بیرونی، کش و تبدیل تصاویر است.



شکل 4.1: معماری کلان مدل پیشرفته

4.2 اصل مالکیت داده

یک تصمیم طراحی بسیار مهم در مدل پیشرفته، تفکیک داده مالکیت کاربر از کش ارائه‌دهند» است: PostgreSQL سورس اف تروث برای داده کاربر است. اطلاعات TMDb در جدول‌های رسانه/فصل/قسمت به‌عنوان کش پایدار ذخیره می‌شود و قابل تازه‌سازی است. Redis برای داده کوتاه‌عمر مانند ترتیب نتایج صفحه اصلی یا نسخه پشتیبان جست‌وجو استفاده می‌شود و پوستر به فایل WebP بهینه‌شده روی سرور تبدیل و کش می‌شود. این تفکیک اجازه می‌دهد اطلاعات خارجی تازه شوند بدون آن‌که تاریخچه و فعالیت کاربر از بین برود.

4.3 لایه‌های سمت کلاینت پیشرفته

جدول 4.1: ساختار کلاینت در مدل پیشرفته

مسئولیت	مسیر
صفحات و جریان‌های کاربری شامل پنل مدیر	mobile/lib/features/*
وضعیت احراز هویت، بازیابی نشست، بیومتریک، ورود و خروج	features/auth/auth_controller.dart
قرارداد سطح کاربردی برای UI و نگاشت JSON به مدل‌های Dart	features/shared/repository.dart
Dio، توکن، تمدید خودکار، پین گواهی، تکرارناپذیری و حذف درخواست GET تکراری	core/api/api_client.dart
ذخیره token در access/refresh flutter_secure_storage	core/auth/token_store.dart
آدرس API، fingerprint، timeout و size page	core/config/app_config.dart
تبدیل خطای شبکه HTTP به پیام مناسب کاربر	core/errors/api_exception.dart

4.4 لایه‌های سمت سرور

جدول 4.2: ساختار بک‌اند در مدل پیشرفته

مسئولیت	مسیر
ساخت برنامه، CORS، middleware، route، health و ready	backend/app/main.py
قرارداد HTTP و endpointهای نسخه اول	backend/app/api/v1/*
اعتبارسنجی ورودی/خروجی با Pydantic	backend/app/schemas/*
امنیت، dependencyها، خطاها، limit rate و idempotency	backend/app/core/*
منطق کسب‌وکار و تراکنش‌های داده	backend/app/services/*
انتزاع ارائه‌دهنده و آداپتور TMDB	backend/app/services/providers/*
مدل‌های SQLAlchemy	backend/app/models.py
مهاجرت پایگاه داده	backend/alembic/*

4.5 چرخه یک درخواست در بک‌اند



شکل 4.2: چرخه پردازش درخواست در مدل پیشرفته

4.6 انتزاع ارائه‌دهنده اطلاعات رسانه‌ای

کلاس انتزاعی `backend/app/services/providers/base.py` چهار عملیات اصلی را تعریف می‌کند: جست‌وجو، دریافت جزئیات عنوان، دریافت قسمت‌ها و بخش‌های صفحه اصلی. بنابراین لایه سرویس نباید جزئیات HTTP سرویس بیرونی را بداند. `factory.py` در نسخه فعلی `TMDBProvider` را انتخاب می‌کند.

این طراحی یک مزیت معماری مهم دارد: اگر در آینده ارائه‌دهنده دیگری اضافه شود، بهتر است آداپتور جدید همان قرارداد `MediaProvider` را پیاده‌سازی کند و بخش عمده API و UI بدون تغییر باقی بماند.

4.7 نحوه استفاده از TMDB در سرور

آدایپتور `backend/app/services/providers/tmdb.py` درخواست‌های واقعی TMDB v3 را می‌سازد، خطاهای `401/403/404/429/5xx` و خطاهای شبکه را به خطاهای دامنه تبدیل می‌کند، سپس پاسخ خام را به ساختار خنثی از ارائه‌دهنده نگاشت می‌کند. خروجی‌هایی مانند `media_key`، عنوان، نوع، بازیگران، کارگردان، ژانرها، امتیاز ارائه‌دهنده، شناسه خارجی IMDb و اطلاعات سریال از این آدایپتور وارد سرویس رسانه می‌شوند.

4.8 کش چندلایه در مدل پیشرفته

جست‌وجو: حالت پیش‌فرض `TMDB_SEARCH_LIVE=true` است؛ درخواست زنده اولویت دارد و `Redis/` داده محلی عمدتاً `fallback` است.

جزئیات: رکورد کامل رسانه در PostgreSQL همراه `cached_at` نگهداری و بر اساس TTL تازه‌سنجی می‌شود.

فصل/قسمت: اطلاعات قسمت‌ها نیز TTL پایگاه داده دارد.

صفحه اصلی: کلیدهای بخش‌ها در Redis کش و خود رسانه‌ها به‌صورت پایدار در PostgreSQL `upsert` می‌شوند.

پوستر: اولین بار از دامنه مجاز TMDB دریافت، محدودیت اندازه و نوع بررسی، به WebP تبدیل/کوچک و روی دیسک ذخیره می‌شود. دفعات بعد فایل محلی پاسخ داده می‌شود.

4.9 پنل مدیریت

مدل پیشرفته مسیرهای مدیریتی و رابط Flutter مخصوص مدیر دارد. مدیر می‌تواند آمار سیستم را ببیند، کاربران را جست‌وجو و نقش/فعال‌بودن را تغییر دهد، گزارش‌ها را بررسی کند، نظر نامناسب را حذف کند و کش رسانه را مدیریت کند. نسخه به‌روزشده یک اصلاح مهم دارد: «پاک‌سازی کش رسانه» دیگر ردیف Media را حذف نمی‌کند؛ بلکه تازگی داده، پوستر مشتق‌شده و کش صفحه اصلی را بی‌اعتبار می‌کند تا امتیاز، نظر، علاقه‌مندی، فهرست و تاریخچه کاربر حفظ شوند.

فصل 5

مستندات API مدل پیشرفته

5.1 اصول قرارداد API

API نسخه‌بندی شده و مسیر اصلی آن `/api/v1` است. روترها بر اساس حوزه به احراز هویت، کاربران، رسانه، کتابخانه شخصی، فهرست‌ها و مدیریت تقسیم شده‌اند. FastAPI با `response_model` و مدل‌های Pydantic قرارداد داده را تولید می‌کند و Swagger در `/docs` و مشخصات OpenAPI در `openapi.json` قابل دسترسی است.

5.2 آدرس پایه و پیکربندی کلاینت

آدرس پیش‌فرض production در کد فعلی:

```
https://31.57.118.82/api/v1
```

در زمان اجرا/ساخت بهتر است آدرس و fingerprint گواهی صریحاً داده شوند:

```
flutter run \
--dart-define="API_BASE_URL=https://31.57.118.82/api/v1" \
--dart-define="CERT_SHA256=<sha256 fingerprint>"
```

ارجاع پیاده‌سازی: `docs/API_CONNECTION.md`

5.3 احراز هویت API

پس از ورود یا ثبت‌نام، سرور دو توکن می‌دهد: `access token` کوتاه‌عمر و `refresh token` با نشست ۳۰ روزه. درخواست‌های محافظت‌شده هدر زیر را دارند:

```
Authorization: Bearer <access-token>
```

کلاینت token access را خودکار به درخواست اضافه می‌کند. اگر پاسخ 401 باشد و درخواست قبلاً retry نشده باشد، فقط یک عملیات refresh همزمان اجرا می‌شود، توکن‌های جدید ذخیره می‌شوند و درخواست اصلی یک بار تکرار می‌شود.

5.4 صفحه‌بندی و فیلتر

endpointهای لیستی مانند جست‌وجو، نظر، کتابخانه و فهرست کاربران از page و page_size استفاده می‌کنند. برای جست‌وجوی رسانه پارامترهای زیر موجود است:

```
q, media_type, actor, director, genre, year, page, page_size
```

media_type فقط movie یا series است؛ سال و اندازه صفحه نیز در سمت سرور محدود می‌شوند.

5.5 قالب خطای استاندارد

خطاهای برنامه در یک envelope یکنواخت برگردانده می‌شوند:

```
{
  "error": {
    "status_code": 422,
    "code": "validation_error",
    "message": "Invalid input",
    "detail": "One or more request fields failed validation.",
    "fields": {"email": ["invalid value"]},
    "request_id": "..."
  }
}
```

request_id در middleware ساخته می‌شود و در پاسخ نیز با X-Request-ID قرار می‌گیرد تا اشکال‌زدایی سمت سرور ممکن باشد.

5.6 Idempotency-Key

برای برخی عملیات تغییر وضعیت، کلاینت هدر Idempotency-Key تولید می‌کند. سرور پاسخ قبلی را بر اساس کاربر، کلید، مدت و مسیر تا ۲۴ ساعت نگهداری می‌کند و در retry همان نتیجه را برمی‌گرداند. این سازوکار در مسیرهای ثبت نظر، وضعیت کتابخانه، امتیاز و قسمت دیده‌شده استفاده شده است.

5.7 تصاویر

کلاینت پیشرفته نباید URL مستقیم TMDB CDN را مصرف کند. پاسخ رسانه، مسیر هم‌دامنه‌ای مانند `ApiClient.getSameOriginBytes()` می‌دهد. `/api/v1/media/{media_id}/poster` فقط آدرس با همان `scheme/host/port` سرور API را می‌پذیرد و همان کانال پین شده TLS را برای تصویر استفاده می‌کند.

5.8 فهرست کامل endpointها

در ادامه قرارداد عملیاتی نسخه فعلی API آمده است. «اختیاری» یعنی مهمان می‌تواند استفاده کند ولی اگر توکن معتبر باشد پاسخ می‌تواند شامل وضعیت شخصی کاربر شود.

جدول 5.1: فهرست endpointهای مدل پیشرفته

روش	مسیر	دسترسی	کارکرد
POST	<code>/api/v1/auth/register</code>	عمومی	ثبت نام multipart همراه تصویر اختیاری
POST	<code>/api/v1/auth/register-json</code>	عمومی	ثبت نام با JSON
POST	<code>/api/v1/auth/login</code>	عمومی	ورود و صدور <code>access/refresh token</code>
POST	<code>/api/v1/auth/refresh</code>	عمومی با <code>re-fresh</code>	چرخش <code>refresh token</code> و صدور توکن جدید
POST	<code>/api/v1/auth/logout</code>	عمومی با <code>re-fresh</code>	ابطال نشست <code>refresh</code>
POST	<code>/api/v1/auth/password/forgot</code>	عمومی	ایجاد توکن بازیابی و ارسال از مسیر <code>mail/console</code>
POST	<code>/api/v1/auth/password/reset</code>	عمومی	تعویض رمز با توکن یک بار مصرف
GET	<code>/api/v1/users/me</code>	کاربر	دریافت پروفایل
PATCH	<code>/api/v1/users/me</code>	کاربر	ویرایش پروفایل
POST	<code>/api/v1/users/me/avatar</code>	کاربر	بارگذاری آواتار
DELETE	<code>/api/v1/users/me/avatar</code>	کاربر	حذف آواتار

روش	مسیر	دسترسی	کارکرد
GET	/api/v1/users/me/stats	کاربر	آمار فعالیت کاربر
GET	/api/v1/users/me/activity	کاربر	تاریخچه فعالیت
GET	/api/v1/media/provider-status	عمومی	وضعیت ارائه‌دهنده TMDB و تنظیمات search live
GET	/api/v1/media/home	اختیاری	بخش‌های صفحه اصلی و پیشنهادات
GET	/api/v1/media/search	عمومی	جست‌وجو و فیلتر فیلم/سریال
GET	/api/v1/media/{media_id}/poster	عمومی	پوستر WebP کش‌شده از سرور
GET	/api/v1/media/{media_id}	اختیاری	جزئیات کامل رسانه
GET	/api/v1/media/{media_id}/seasons	اختیاری	فصل‌ها و قسمت‌ها
GET	/api/v1/media/{media_id}/comments	اختیاری	نظرات صفحه‌بندی‌شده
POST	/api/v1/media/{media_id}/comments	کاربر	ثبت نظر جدید؛ پشتیبانی Idempotency-Key
PATCH	/api/v1/comments/{comment_id}	مالک/مدیر	ویرایش نظر
DELETE	/api/v1/comments/{comment_id}	مالک/مدیر	حذف نظر
POST	/api/v1/comments/{comment_id}/reports	کاربر	گزارش نظر
GET	/api/v1/me/library	کاربر	کتابخانه شخصی با فیلتر وضعیت
PUT	/api/v1/me/library/{media_id}	کاربر	تنظیم وضعیت تماشا
DELETE	/api/v1/me/library/{media_id}	کاربر	حذف از کتابخانه
PUT	/api/v1/me/favorites/{media_id}	کاربر	افزودن به علاقه‌مندی

روش	مسیر	دسترسی	کارکرد
	/api/v1/me/favorites/ DELETE {media_id}	کاربر	حذف از علاقه‌مندی
PUT	/api/v1/me/ratings/ {media_id}	کاربر	ثبت/ویرایش امتیاز ۱ تا ۵
	/api/v1/me/ratings/ DELETE {media_id}	کاربر	حذف امتیاز
PUT	/api/v1/me/episodes/ {episode_id}	کاربر	علامت دیده‌شدن قسمت
GET	/api/v1/me/series/ {media_id}/progress	کاربر	درصد پیشرفت سریال
GET	/api/v1/me/lists	کاربر	فهرست‌های شخصی
POST	/api/v1/me/lists	کاربر	ساخت فهرست
GET	/api/v1/me/lists/ {list_id}	مالک/مدیر	جزئیات فهرست
PATCH	/api/v1/me/lists/ {list_id}	مالک/مدیر	ویرایش فهرست
	/api/v1/me/lists/ DELETE {list_id}	مالک/مدیر	حذف فهرست
POST	/api/v1/me/lists/ {list_id}/items	مالک/مدیر	افزودن رسانه به فهرست
	/api/v1/me/lists/ DELETE {list_id}/items/ {media_id}	مالک/مدیر	حذف رسانه از فهرست
GET	/api/v1/admin/stats	مدیر	آمار کلی سیستم
GET	/api/v1/admin/users	مدیر	جست‌وجو/صفحه‌بندی کاربران
PATCH	/api/v1/admin/users/ {user_id}	مدیر	تغییر نقش یا فعال‌بودن کاربر
GET	/api/v1/admin/reports	مدیر	مشاهده گزارش‌ها
PATCH	/api/v1/admin/reports/ {report_id}	مدیر	حل/رد گزارش و ثبت یادداشت
	/api/v1/admin/ DELETE comments/{comment_id}	مدیر	حذف نظر نامناسب

روش	مسیر	دسترسی	کارکرد
GET	/api/v1/admin/media	مدیر	مشاهده رسانه‌های کش‌شده
PATCH	/api/v1/admin/media/{media_id}	مدیر	ویرایش داده کش‌شده
DELETE	/api/v1/admin/media/{media_id}	مدیر	پاک‌سازی امن کش رسانه بدون حذف فعالیت کاربر
POST	/api/v1/admin/media/{media_id}/refresh	مدیر	تازه‌سازی رسانه از TMDb
GET	/health	عمومی	سلامت پردازش برنامه
GET	/ready	عمومی	آمادگی سرویس و دسترسی پایگاه داده

5.9 نمونه‌های درخواست و پاسخ

5.9.1 ورود

```
POST /api/v1/auth/login
Content-Type: application/json
```

```
{
  "email": "user@example.com",
  "password": "StrongPass123"
}
```

پاسخ شامل access_token، refresh_token، زمان انقضا و اطلاعات کاربر است و در کلاینت به AuthResult نگاشت می‌شود.

5.9.2 جست‌وجو

```
GET /api/v1/media/search?q=Interstellar&media_type=movie&page=1&
page_size=20
```

نمونه یک آیتم پاسخ:

```
{
  "media_id": "tmdb-movie-157336",
  "media_type": "movie",
  ...
}
```

```
"title": "Interstellar",
"poster_url": "/api/v1/media/tmdb-movie-157336/poster",
"provider_rating": 8.5
}
```

5.9.3 ثبت امتیاز

```
PUT /api/v1/me/ratings/tmdb-movie-157336
Authorization: Bearer <token>
Idempotency-Key: <uuid>
Content-Type: application/json

{"score": 5}
```

سرور یک رکورد یکتا برای کاربر/رسانه ایجاد یا به‌روزرسانی می‌کند و توزیع امتیازها را برمی‌گرداند.

5.9.4 ثبت نظر اسپویلر

```
POST /api/v1/media/tmdb-movie-157336/comments
Authorization: Bearer <token>
Idempotency-Key: <uuid>
Content-Type: application/json

{"text": "My comment", "is_spoiler": true}
```

در UI، متن نظر اسپویلر در ابتدا پنهان می‌ماند تا کاربر آن را آشکار کند.

فصل 6

اتصال اپلیکیشن Flutter به سرور در مدل پیشرفته

6.1 اصل ارتباط

در مدل پیشرفته تنها مقصد شبکه‌ای کلاینت، سرور CineTrack است. توکن TMDb، URL خام پوستر ارائه‌دهنده و منطق دسترسی به پایگاه داده به موبایل داده نمی‌شوند.

6.2 پیکربندی زمان ساخت

در `mobile/lib/core/config/app_config.dart` آدرس API و `fingerprint` را از `dart-define` می‌خواند. در `production`، URL باید HTTPS باشد و `fingerprint` خالی قابل قبول نیست؛ حالت توسعه گیرین شده فقط با پرچم صریح مخصوص توسعه مجاز می‌شود.

6.3 پین کردن گواهی TLS

در `api_client.dart` برای Dio یک `HttpClient` با `SecurityContext(withTrustedRoots: false)` ساخته می‌شود. گواهی دریافتی به بایت‌های DER تبدیل، SHA-256 آن محاسبه و با `fin-gerprint` تنظیم شده مقایسه می‌شود. بنابراین حتی اگر دستگاه یک CA دیگر را معتبر بداند، اتصال فقط در صورت تطابق `fingerprint` پذیرفته می‌شود.

6.4 ذخیره امن توکن

در `token_store.dart` از `flutter_secure_storage` و در اندروید از `encryptedSharedPreferences` استفاده می‌کند. `access` و `token refresh` در حافظه عادی یا فایل ساده برنامه ذخیره نمی‌شوند.

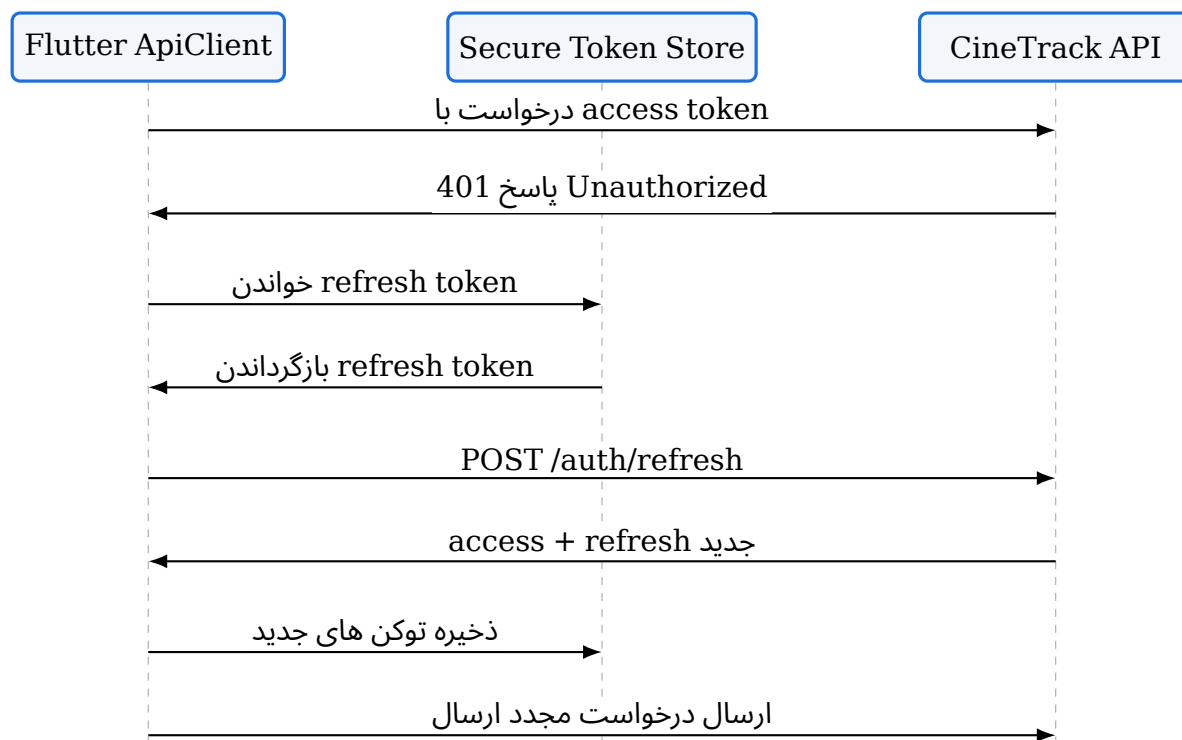
6.5 افزودن خودکار Authorization

Interceptor قبل از هر درخواست token access را می‌خواند و در صورت وجود، هدر Bearer می‌گذارد. این باعث می‌شود Repository و صفحات مجبور نباشند در هر متد منطق توکن را تکرار کنند.

6.6 تمدید خودکار نشست

اگر API پاسخ 401 دهد:

1. کلاینت بررسی می‌کند درخواست login/refresh نباشد و قبلاً retry نشده باشد.
2. متد `_refreshTokens()` اجرا می‌شود. متغیر `_refreshFuture` باعث می‌شود چند 401 همزمان تنها یک refresh واقعی ایجاد کنند.
3. token refresh به `/auth/refresh` ارسال می‌شود.
4. توکن‌های جدید به `storage secure` نوشته می‌شوند.
5. درخواست اولیه با token access جدید یک بار تکرار می‌شود.
6. اگر refresh شکست بخورد، توکن‌ها پاک می‌شوند و برنامه به سمت ورود مجدد هدایت می‌شود.



شکل 6.1: جریان نوسازی خودکار نشست در کلاینت

6.7 Repository در کلاینت پیشرفته

در مدل پیشرفته، Repository بسیار سبک‌تر از مدل ساده است؛ زیرا SQL، کش ارائه‌دهنده و منطق اصلی سمت سرور منتقل شده‌اند. برای مثال `search()` فقط پارامترها را می‌سازد و `/media/search` را فراخوانی می‌کند. `rate()` یک PUT با امتیاز به `/me/ratings/{media_id}` می‌زند. `customLists()` مستقیماً از `/me/lists` مدل می‌سازد. متدهای `admin` فقط وقتی نقش کاربر مدیر است در UI قابل استفاده‌اند، و سرور نیز مستقل از UI مجوز را کنترل می‌کند.

6.8 دریافت امن تصویر

`getSameOriginBytes()` فقط URL‌هایی را قبول می‌کند که `scheme`، `host` و `port` آن‌ها با API یکی باشد. سپس تصویر از همان `Dio` پین‌شده دریافت و تا ۶۴ آیتم در حافظه کش می‌شود. این طراحی مانع آن می‌شود که داده سرور، کلاینت را به دانلود تصویر از دامنه دلخواه هدایت کند.

فصل 7

طراحی پایگاه داده و مالکیت اطلاعات

در نسخه ساده CineTrack، داده‌ها عمدتاً در خود دستگاه کاربر نگهداری می‌شوند و در نتیجه مالکیت و مدیریت اطلاعات نیز در سطح کلاینت متمرکز است. اطلاعات حساب و پروفایل، وضعیت تماشای فیلم و سریال، امتیازها، نظرها و کتابخانه شخصی کاربر در پایگاه داده محلی SQLite ذخیره می‌شوند و اطلاعات نشست نیز در حافظه محلی برنامه، مانند SharedPreferences، نگهداری می‌شود. داده‌های مربوط به فیلم‌ها و سریال‌ها مستقیماً از TMDb دریافت شده و بخشی از آن‌ها برای استفاده مجدد در حافظه یا پایگاه داده محلی کش می‌شوند. این مدل برای یک برنامه تک‌دستگاهی ساده و کم‌هزینه مناسب است، اما از آنجا که داده‌ها به همان دستگاه وابسته‌اند، همگام‌سازی میان چند دستگاه، مدیریت متمرکز کاربران و بازیابی اطلاعات در صورت از دست رفتن دستگاه با محدودیت مواجه می‌شود.

در نسخه پیشرفته، ساختار مالکیت داده به صورت اساسی تغییر کرده و بخش اصلی اطلاعات از کلاینت به سرور منتقل شده است. PostgreSQL به عنوان پایگاه داده مرکزی، اطلاعات حساب کاربران، پروفایل‌ها، وضعیت تماشا، امتیازها، نظرها، فهرست‌های شخصی و سایر داده‌های دائمی را نگهداری می‌کند. نشست کاربر نیز دیگر صرفاً یک وضعیت محلی نیست؛ Token Refresh در سمت سرور ثبت می‌شود و کلاینت فقط توکن‌های لازم را در Storage Secure نگهداری می‌کند. اطلاعات دریافت‌شده از TMDb نیز به صورت کش پایدار در PostgreSQL ذخیره می‌شود و برای داده‌های موقتی‌تر یا پرتکرار از Redis استفاده می‌شود. پوسترها نیز پس از دریافت و پردازش، به صورت WebP در کش سرور ذخیره شده و سپس توسط کلاینت دریافت می‌شوند. این معماری باعث می‌شود داده‌های کاربر مستقل از دستگاه باشند و امکان استفاده از چند دستگاه، مدیریت متمرکز، کنترل دسترسی و پشتیبان‌گیری از اطلاعات فراهم شود.

مدل داده نسخه پیشرفته در فایل `backend/app/models.py` و با استفاده از SQLAlchemy تعریف شده است. هسته این مدل از جدول `users` آغاز می‌شود که اطلاعات اصلی کاربران مانند شناسه، نام، نام کاربری، ایمیل، مقدار هش‌شده رمز عبور، نقش کاربر و وضعیت فعال یا غیرفعال بودن را نگهداری می‌کند. نام کاربری و ایمیل دارای قید یکتایی هستند تا از ایجاد حساب‌های تکراری جلوگیری شود. در کنار این جدول، `refresh_tokens` برای مدیریت نشست‌های بلندمدت کاربران استفاده می‌شود و اطلاعاتی مانند شناسه یکتای توکن، زمان انقضا، زمان ابطال و مشخصات دستگاه را ثبت می‌کند. همچنین `password_reset_tokens` برای مدیریت امن فرایند بازیابی رمز عبور

طراحی شده و تنها هش توکن یک بار مصرف، زمان انقضا و زمان استفاده از آن را نگهداری می‌کند. اطلاعات رسانه‌ها در جدول `media` متمرکز شده است. این جدول به صورت مشترک برای فیلم و سریال استفاده می‌شود و علاوه بر شناسه داخلی، اطلاعاتی مانند `provider`، شناسه `TMDB`، شناسه خارجی `IMDb`، عنوان، متادیتای اصلی، داده خام دریافتی از سرویس خارجی و زمان آخرین به‌روزرسانی کش را ذخیره می‌کند. این طراحی از ایجاد جداول جداگانه و تکراری برای فیلم و سریال جلوگیری می‌کند و امکان توسعه ساده‌تر لایه رسانه را فراهم می‌سازد. برای سریال‌ها، ساختار فصل و قسمت در دو جدول `seasons` و `episodes` مدل‌سازی شده است. هر فصل به یک رسانه مشخص تعلق دارد و شماره فصل در محدوده همان رسانه یکتا است. قسمت‌ها نیز به یک فصل مرتبط می‌شوند و شماره هر قسمت در فصل مربوطه دارای قید یکتایی است.

داده‌های مربوط به تعامل کاربر با رسانه‌ها در چند جدول مستقل نگهداری می‌شوند. جدول `user_media` وضعیت کلی تماشای یک رسانه توسط کاربر را ذخیره می‌کند و با قید `unique(user, media)` تضمین می‌شود که برای هر کاربر و هر رسانه تنها یک وضعیت اصلی وجود داشته باشد. مقدار وضعیت نیز با `constraint CHECK` به گزینه‌های مجاز محدود می‌شود. جدول `user_episodes` برای ثبت مشاهده شدن قسمت‌های سریال استفاده می‌شود و هر ترکیب کاربر و قسمت فقط یک بار قابل ثبت است. امتیازها در جدول `ratings` ذخیره می‌شوند و علاوه بر یکتایی ترکیب کاربر و رسانه، محدوده امتیاز نیز در سطح پایگاه داده کنترل می‌شود. به این ترتیب حتی اگر اعتبارسنجی لایه `API` دور زده شود، پایگاه داده از ثبت مقدار نامعتبر جلوگیری می‌کند.

نظرات کاربران در جدول `comments` نگهداری می‌شوند. علاوه بر متن نظر، این جدول اطلاعاتی مانند `spoiler` بودن نظر، وضعیت `hidden` و یادداشت مربوط به `moderation` را نیز در خود دارد تا امکان مدیریت و کنترل محتوای کاربران فراهم باشد. علاقه‌مندی‌ها در جدول `favorites` ذخیره می‌شوند و با قید یکتا از ثبت چندباره یک رسانه برای یک کاربر جلوگیری می‌شود. برای فهرست‌های شخصی نیز دو جدول `custom_lists` و `custom_list_items` در نظر گرفته شده است؛ جدول اول مشخصات فهرست، مالک آن و عمومی یا خصوصی بودن فهرست را نگهداری می‌کند و جدول دوم ارتباط رسانه‌ها با هر فهرست و ترتیب آن‌ها را ثبت می‌کند.

برای قابلیت‌های مدیریتی و کنترلی، جدول `reports` امکان ثبت گزارش درباره نظرها یا محتوای کاربران را فراهم می‌کند و اطلاعات مربوط به وضعیت بررسی و توضیح نتیجه رسیدگی را نگهداری می‌کند. علاوه بر این، جدول `idempotency_records` برای جلوگیری از اجرای چندباره برخی عملیات حساس طراحی شده است. این جدول نتیجه قبلی یک درخواست را بر اساس ترکیب کاربر، کلید `idempotency`، متد `HTTP` و مسیر درخواست ثبت می‌کند تا در صورت تکرار همان درخواست، پاسخ قبلی قابل بازیابی باشد. جدول `audit_logs` نیز به عنوان مدل پیش‌بینی‌شده برای ثبت رخدادهای مدیریتی و ممیزی در ساختار پایگاه داده وجود دارد و می‌تواند در توسعه‌های بعدی برای ثبت تغییرات مهم، اقدامات مدیر و رویدادهای امنیتی مورد استفاده قرار گیرد.

فصل 8

امنیت

8.1 مدل ساده

امنیت مدل ساده متناسب با معماری محلی است. رمز plaintext ذخیره نمی‌شود و با PBKDF2 + salt هش می‌شود. تمام endpointهای TMDB از HTTPS استفاده می‌کنند. کوئری‌های SQLite پارامتری هستند. نشست محلی تاریخ انقضا دارد و می‌تواند با بیومتریک محافظت شود

8.2 مدل پیشرفته

8.2.1 رمز و نشست

سرور از Argon2 با پارامترهای هزینه حافظه/زمان برای هش رمز استفاده می‌کند. to-access tokenهای JWT کوتاه‌عمر هستند و token refresh دارای tjt، زمان انقضا و امکان ابطال است. session refresh logout revoke را می‌کند.

8.2.2 مجوز

Dependencyهای AdminUser و CurrentUser هویت و نقش را از سمت سرور تعیین می‌کنند. مالکیت نظر و فهرست نیز در سرویس بررسی می‌شود. در نتیجه مخفی‌کردن دکمه در UI تنها لایه راحتی است و امنیت به آن وابسته نیست.

8.2.3 انتقال امن

HTTP Nginx را به HTTPS هدایت می‌کند، TLS 1.2/1.3 و HSTS را فعال می‌کند و Flutter نیز fingerprint گواهی را پین می‌کند.

8.2.4 کلید سرویس بیرونی

TMDB_READ_ACCESS_TOKEN یا TMDB_API_KEY فقط در تنظیمات سرور هستند. کلاینت API کلید TMDB را دریافت نمی‌کند.

8.2.5 اعتبارسنجی و ورودی مخرب

مدل‌های Pydantic طول و قالب ایمیل، username، رمز، امتیاز، نظر، سال و pagination را اعتبارسنجی می‌کنند. SQLAlchemy از parameter bind استفاده می‌کند. فایل آواتار بر اساس نوع/اندازه و decode واقعی تصویر کنترل و دوباره encode می‌شود.

8.2.6 محافظت SSRF در پوستر

سرور تنها URL با HTTPS و hostname مطابق مبدا تصویر TMDB را می‌پذیرد، اندازه دانلود را محدود می‌کند، نوع محتوا را بررسی و تصویر را با decode Pillow و به WebP بازنویسی می‌کند. بنابراین proxy poster مسیر عمومی برای خواندن آدرس‌های داخلی سرور نیست.

8.2.7 limiting Rate و هدرهای امنیتی

میان‌افزار داخلی بر اساس IP تعداد درخواست در دقیقه را محدود می‌کند و Nginx نیز در لبه محدودسازی دارد. پاسخ‌ها هدرهایی مانند X-Content-Type-Options, X-Frame-Options, Referrer-Policy و در HSTS production دریافت می‌کنند.

فصل 9

قابلیت اطمینان، کارایی، مقیاس‌پذیری و مصرف منابع

9.1 قابلیت اطمینان

مدل ساده به دلیل محلی بودن داده کاربر در برابر قطع سرور اختصاصی مصون است و در قطع TMDB می‌تواند از cache stale استفاده کند. در مدل پیشرفته، PostgreSQL منبع پایدار فعالیت کاربر است و خرابی TMDB نباید ورود، پروفایل یا داده‌های قبلی را از کار بیندازد. سرویس رسانه در صورت عدم دسترسی provider می‌تواند جزئیات/قسمت کش‌شده را بازگرداند.

9.2 مقیاس‌پذیری

مدل پیشرفته مقیاس‌پذیرتر است چون کاربران چند دستگاه داده مشترک دارند، PostgreSQL ایندکس و pool دارد، Redis قابل اشتراک است و stateless API با JWT کوتاه‌عمر طراحی شده است. با این حال چند نقطه برای مقیاس بالا نیاز به بهبود دارد: بعضی مسیرهای کتابخانه و پیشنهادها حجم زیادی از ردیف‌ها را قبل از صفحه‌بندی/مرتب‌سازی در حافظه برنامه می‌خوانند و deployment فعلی برای VPS کوچک با یک worker تنظیم شده است.

9.3 مصرف باتری و اینترنت

هیچ polling دائمی در جریان عادی دیده نمی‌شود. تصاویر فشرده و کش می‌شوند و درخواست‌های هم‌زمان یکسان حذف می‌شوند. در مدل پیشرفته، نکته قابل توجه این است که `TMDB_SEARCH_LIVE=true` باعث می‌شود جست‌وجوی تکراری عادی دوباره به TMDB برسد؛ کش جست‌وجو بیشتر fallback است. اگر هدف کاهش حداکثری مصرف اینترنت باشد، TTL کوتاه فعال برای نتیجه جست‌وجو مفید است.

فصل 10

مقایسه مدل ساده و پیشرفته

در نسخه ساده معماری سامانه بر مبنای اجرای عمده منطق برنامه در خود کلاینت شکل گرفته است. به این معنا که اپلیکیشن Flutter مستقیماً با سرویس TMDb ارتباط برقرار می‌کند و اطلاعات مربوط به کاربران، وضعیت تماشا، امتیازها، نظرها و سایر داده‌های شخصی را در پایگاه داده محلی SQLite همان دستگاه نگهداری می‌کند. احراز هویت نیز به صورت محلی انجام می‌شود و نشست کاربر در حافظه دستگاه مدیریت می‌گردد. این رویکرد از نظر پیاده‌سازی و استقرار ساده‌تر است و هزینه عملیاتی بسیار کمی دارد، زیرا به سرور مستقل نیاز ندارد. اما در هر صورت محدودیت‌هایی مانند نبود همگام‌سازی میان چند دستگاه، قرار گرفتن کلید TMDb در سمت کلاینت، نبود مدیریت متمرکز کاربران و وابستگی بخش عمده منطق کسب‌وکار به برنامه موبایل را به همراه دارد. در چنین معماری‌ای، Repository کلاینت عملاً بخش بزرگی از مسئولیت‌های سیستم را بر عهده دارد و عملیات‌هایی مانند ورود، جست‌وجو، ثبت امتیاز، نظر و وضعیت تماشا عمدتاً در لایه Dart و پایگاه داده محلی مدیریت می‌شوند.

در نسخه پیشرفته، این مسئولیت‌ها به شکل محسوسی از کلاینت به سرور منتقل شده‌اند و معماری به یک مدل واقعی کلاینت-سرور تبدیل شده است. اپلیکیشن Flutter دیگر مستقیماً با TMDb یا پایگاه داده اصلی کار نمی‌کند، بلکه تمام عملیات اصلی را از طریق API اختصاصی CineTrack انجام می‌دهد؛ سپس سرور FastAPI وظیفه ارتباط با TMDb، اعتبارسنجی داده‌ها، اجرای قواعد سیستم، کنترل دسترسی، مدیریت نشست‌ها و ذخیره‌سازی اطلاعات در PostgreSQL را بر عهده می‌گیرد. استفاده از JWT، Token Refresh، Argon2، RBAC، Redis، کش تصاویر، HTTPS و مستندسازی Swagger/OpenAPI نیز باعث شده است امنیت، قابلیت اطمینان، مدیریت متمرکز داده و امکان استفاده هم‌زمان از چند دستگاه به شکل قابل توجهی بهبود یابد. در نتیجه، اگرچه این معماری به استقرار و نگهداری سرور، پایگاه داده و زیرساخت شبکه نیاز دارد و از نظر عملیاتی پیچیده‌تر است، اما برای یک سامانه چندکاربره و قابل توسعه انتخاب مناسب‌تری محسوب می‌شود؛ در این مدل، Repository کلاینت دیگر سیستم نیست، بلکه صرفاً لایه‌ای برای مصرف سرویس‌های ارائه شده توسط سرور است.

فصل 11

محدودیت ها و چالش ها

11.1 TMDB به جای IMDb

ما ناچار شدیم که به جای دیتابیس اصلی imdb از tmdb استفاده کنیم زیرا اتصال به ای ام دی بی بسیار هزینه بر بود ولی این بهترین جایگزین ممکن بود که از آن بهره ببریم.

11.2 کش جست و جو

با `TMDB_SEARCH_LIVE=true` جست و جوی تکراری حتی در فاصله کوتاه معمولا دوباره به provider می رود. و دلیل اصلی این امر این بود که احتمال داشت تغییر صورت بگیرد.

فصل 12

جمع‌بندی

مدل ساده و پیشرفته دو پاسخ متفاوت به یک دامنه مشترک‌اند. مدل ساده نشان می‌دهد چگونه می‌توان با یک برنامه Flutter، سرویس اطلاعات بیرونی و SQLite محلی، بخش بزرگی از تجربه کاربری را بدون سرور مستقل پیاده‌سازی کرد. این مدل برای آموزش، نمونه‌سازی، اجرای تک‌دستگاه و کاهش هزینه عملیاتی بسیار مناسب است؛ اما همگام‌سازی، مدیریت مرکزی و محافظت از کلید سرویس محدودیت ذاتی آن هستند.

مدل پیشرفته سامانه را به یک معماری واقعی کلاينت/سرور تبدیل می‌کند. مسئولیت‌های امنیتی و داده‌ای به FastAPI و PostgreSQL منتقل می‌شوند، Redis و کش فایل هزینه provider را کاهش می‌دهند، Nginx و pinning certificate مسیر انتقال را سخت‌تر می‌کنند، JWT و re-session fresh نشست چنددستگاهی را ممکن می‌سازند و API نسخه‌بندی‌شده قرارداد مشخصی بین Flutter و سرور ایجاد می‌کند. طراحی abstraction provider نیز باعث می‌شود سرویس رسانه نسبت به جزئیات TMDb وابستگی کمتری داشته باشد.

از دید مهندسی، نسخه پیشرفته پایه مناسبی برای یک سامانه قابل استقرار دارد و اصلاح cache delete در نسخه به‌روز اهمیت ویژه‌ای دارد؛ زیرا داده کاربر را از داده cache جدا می‌کند. برای رسیدن به کیفیت production قوی‌تر، مهم‌ترین اقدامات بعدی شامل اتمی‌کردن idempotency، کامل‌ترکردن OpenAPI errors، سخت‌گیری secrets در production، بهبود query‌های بزرگ، cache کوتاه جست‌وجو و شکستن فایل‌های بسیار بزرگ است.

در نهایت، در ارائه دانشگاهی باید تفاوت TMDb/IMDb شفاف بیان شود: معماری خواسته‌شده در مدل ساده و پیشرفته پیاده شده، اما ارائه‌دهنده واقعی اطلاعات در هر دو کدبیس TMDb است.